

Queue, PriorityQueue, and ArrayDeque are all data structures used to store and manage collections of elements in computer programming. Each of these data structures has its specific characteristics and use cases. Let's explore the differences between them:

1. Queue:

A Queue is a basic data structure that follows the First-In-First-Out (FIFO) principle. It means that the first element inserted into the queue will be the first one to be removed. New elements are added to the back (rear) of the queue, and removal happens from the front (head) of the queue. Common operations supported by a queue are:

- Enqueue (add/offer): Adds an element to the back of the queue.
- Dequeue (remove/poll): Removes the front element from the queue.
- Peek: Returns the front element without removing it.

Queue is typically implemented using various data structures like linked lists, arrays, or dynamic arrays (e.g., ArrayList in Java). It's useful in scenarios where processing elements in the order of their arrival is required, like task scheduling, breadth-first search algorithms, etc.

2. PriorityQueue:

A PriorityQueue is an extension of the regular Queue data structure but with a different characteristic. Instead of following the FIFO principle, a PriorityQueue follows the priority-based order. Elements are dequeued based on their priority, where the element with the highest priority is removed first. Priority can be defined by implementing a comparator or using the natural ordering of the elements (if they are comparable). Common operations supported by a priority queue are:

- Insert (add/offer): Adds an element to the priority queue.
- Extract (remove/poll): Removes the element with the highest priority from the queue.
- Peek: Returns the element with the highest priority without removing it.

A priority queue is typically implemented using various data structures like binary heaps, Fibonacci heaps, or balanced search trees. It's commonly used in tasks where processing elements with higher importance or urgency is necessary, such as scheduling jobs with different priorities, Dijkstra's algorithm, Huffman coding, etc.

3. ArrayDeque:

An ArrayDeque (short for "Array Double-Ended Queue") is a special kind of double-ended queue (deque). It allows adding and removing elements from both ends of the data structure, making it more versatile than a regular queue. Elements can be added and removed from both the front and the back in constant time, which means $O(1)$ time complexity for these operations. Common operations supported by an ArrayDeque are:

- AddFirst: Adds an element to the front of the deque.
- AddLast: Adds an element to the back of the deque.
- RemoveFirst: Removes and returns the element from the front of the deque.
- RemoveLast: Removes and returns the element from the back of the deque.
- PeekFirst: Returns the element at the front without removing it.
- PeekLast: Returns the element at the back without removing it.

ArrayDeque is typically implemented using a resizable array or a circular buffer. It is useful when you need efficient insertion and removal of elements at both ends of the queue, like implementing a stack or dequeuing elements from both ends simultaneously.

In summary, here's a quick overview of the differences:

- Queue: Follows FIFO order for adding and removing elements.
- PriorityQueue: Prioritizes elements based on their priority when adding and removing

.
- **ArrayDeque**: Supports adding and removing elements from both ends, providing efficient **double-ended** queue operations.

Each of these data structures has its specific use cases, and choosing the right one depends on the requirements of your application or algorithm.

```
import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {
        // Example with Integer data
        PriorityQueue<Integer> integerQueue = new PriorityQueue<>();

        // Adding elements to the Integer Priority Queue using add() and offer()
        // methods.
        integerQueue.add(5);
        integerQueue.offer(2);
        integerQueue.add(10);
        integerQueue.offer(7);

        System.out.println("Integer Priority Queue: " + integerQueue);

        // Removing elements from the Integer Priority Queue using remove() and
        // poll() methods.
        int removedInteger1 = integerQueue.remove();
        int removedInteger2 = integerQueue.poll();

        System.out.println("Removed Integer Element 1: " + removedInteger1);
        System.out.println("Removed Integer Element 2: " + removedInteger2);
        System.out.println("Updated Integer Priority Queue: " + integerQueue);

        // Getting the first element from the Integer Priority Queue using element()
        // and peek() methods.
        int firstInteger1 = integerQueue.element();
        int firstInteger2 = integerQueue.peek();

        System.out.println("First Integer Element 1: " + firstInteger1);
        System.out.println("First Integer Element 2: " + firstInteger2);
        System.out.println("Integer Priority Queue after peeking: " + integerQueue);

        System.out.println("\n-----\n");

        // Example with String data
        PriorityQueue<String> stringQueue = new PriorityQueue<>();

        // Adding elements to the String Priority Queue using add() and offer()
        // methods.
        stringQueue.add("Apple");
        stringQueue.offer("Banana");
        stringQueue.add("Orange");
        stringQueue.offer("Grapes");

        System.out.println("String Priority Queue: " + stringQueue);

        // Removing elements from the String Priority Queue using remove() and poll()
        // methods.
        String removedString1 = stringQueue.remove();
        String removedString2 = stringQueue.poll();
```

```

88
89     System.out.println("Removed String Element 1: " + removedString1);
90     System.out.println("Removed String Element 2: " + removedString2);
91     System.out.println("Updated String Priority Queue: " + stringQueue);
92
93     // Getting the first element from the String Priority Queue using element()
    and peek() methods.
94     String firstString1 = stringQueue.element();
95     String firstString2 = stringQueue.peek();
96
97     System.out.println("First String Element 1: " + firstString1);
98     System.out.println("First String Element 2: " + firstString2);
99     System.out.println("String Priority Queue after peeking: " + stringQueue);
100 }
101 }
102 -----
-----
103
104 import java.util.ArrayDeque;
105 import java.util.Collection;
106
107 public class ArrayDequeExample {
108     public static void main(String[] args) {
109         // ArrayDeque creation with String type
110         ArrayDeque<String> stringArrayDeque = new ArrayDeque<>();
111
112         // add(Element e) for String type
113         stringArrayDeque.add("Apple");
114         stringArrayDeque.add("Banana");
115         stringArrayDeque.add("Orange");
116
117         // addAll(Collection<? extends E> c) for String type
118         Collection<String> otherFruits = new ArrayDeque<>();
119         otherFruits.add("Grapes");
120         otherFruits.add("Watermelon");
121         stringArrayDeque.addAll(otherFruits);
122
123         // addFirst(Element e) for String type
124         stringArrayDeque.addFirst("Pineapple");
125
126         // addLast(Element e) for String type
127         stringArrayDeque.addLast("Mango");
128
129         // clear() for String type
130         stringArrayDeque.clear();
131
132         // isEmpty() for String type
133         boolean isStringArrayDequeEmpty = stringArrayDeque.isEmpty();
134         System.out.println("Is String ArrayDeque empty? " + isStringArrayDequeEmpty);
135
136         // offerFirst(Element e) for String type
137         stringArrayDeque.offerFirst("Cherry");
138
139         // offerLast(Element e) for String type
140         stringArrayDeque.offerLast("Pear");
141
142         // peek() for String type
143         String firstStringElement = stringArrayDeque.peek();
144         System.out.println("First String element: " + firstStringElement);

```

```

145
146 // getFirst() for String type
147 String firstStringElementAgain = stringArrayDeque.getFirst();
148 System.out.println("First String element again: " + firstStringElementAgain);
149
150 // getLast() for String type
151 String lastStringElement = stringArrayDeque.getLast();
152 System.out.println("Last String element: " + lastStringElement);
153
154 // remove() for String type
155 stringArrayDeque.remove(); // Removes the first String element
156
157 // removeFirst() for String type
158 stringArrayDeque.removeFirst();
159
160 // removeLast() for String type
161 stringArrayDeque.removeLast();
162
163 // size() for String type
164 int stringArrayDequeSize = stringArrayDeque.size();
165 System.out.println("String ArrayDeque size: " + stringArrayDequeSize);
166
167
168 // ArrayDeque creation with Integer type
169 ArrayDeque<Integer> integerArrayDeque = new ArrayDeque<>();
170
171 // add(Element e) for Integer type
172 integerArrayDeque.add(10);
173 integerArrayDeque.add(20);
174 integerArrayDeque.add(30);
175
176 // addAll(Collection<? extends E> c) for Integer type
177 Collection<Integer> otherNumbers = new ArrayDeque<>();
178 otherNumbers.add(40);
179 otherNumbers.add(50);
180 integerArrayDeque.addAll(otherNumbers);
181
182 // addFirst(Element e) for Integer type
183 integerArrayDeque.addFirst(5);
184
185 // addLast(Element e) for Integer type
186 integerArrayDeque.addLast(60);
187
188 // clear() for Integer type
189 integerArrayDeque.clear();
190
191 // isEmpty() for Integer type
192 boolean isIntegerArrayDequeEmpty = integerArrayDeque.isEmpty();
193 System.out.println("Is Integer ArrayDeque empty? " + isIntegerArrayDequeEmpty);
194
195 // offerFirst(Element e) for Integer type
196 integerArrayDeque.offerFirst(100);
197
198 // offerLast(Element e) for Integer type
199 integerArrayDeque.offerLast(200);
200
201 // peek() for Integer type
202 Integer firstIntegerElement = integerArrayDeque.peek();

```

```
203     System.out.println("First Integer element: " + firstIntegerElement);
204
205     // getFirst() for Integer type
206     Integer firstIntegerElementAgain = integerArrayDeque.getFirst();
207     System.out.println("First Integer element again: " + firstIntegerElementAgain
208 );
209
210     // getLast() for Integer type
211     Integer lastIntegerElement = integerArrayDeque.getLast();
212     System.out.println("Last Integer element: " + lastIntegerElement);
213
214     // remove() for Integer type
215     integerArrayDeque.remove(); // Removes the first Integer element
216
217     // removeFirst() for Integer type
218     integerArrayDeque.removeFirst();
219
220     // removeLast() for Integer type
221     integerArrayDeque.removeLast();
222
223     // size() for Integer type
224     int integerArrayDequeSize = integerArrayDeque.size();
225     System.out.println("Integer ArrayDeque size: " + integerArrayDequeSize);
226 }
227
228
229
```